AsOf

A policy time machine and audit trail for AI assistants, built on IBM Db2

What did the policy say on that date, and what did our AI tell the customer?

IBM Db2 12.1.5 | **LangChain + LangGraph** | **Open source, Apache-2.0** | **Two domains: airline and bank**

Jay Malhotra | solo entry | IBM Db2Hack 2026, TechX Summit | final submission

github.com/JustJay7/asof

Companies own what their AI assistant says



Moffatt v. Air Canada

2024 BCCRT 149, decided 14 Feb 2024

The airline's website chatbot told a grieving customer he could claim a bereavement fare after travelling. The airline's own policy said he could not. The tribunal held the airline liable and rejected the argument that the chatbot was responsible for its own words.

CA$812

ordered in damages, interest and fees, over one chatbot answer



The company owns every statement

The tribunal rejected the defence that the correct policy was published on another page.



Policies change, answers drift

Refund windows, fees and eligibility rules change. Bots, cached FAQ answers and help articles keep repeating the old version.



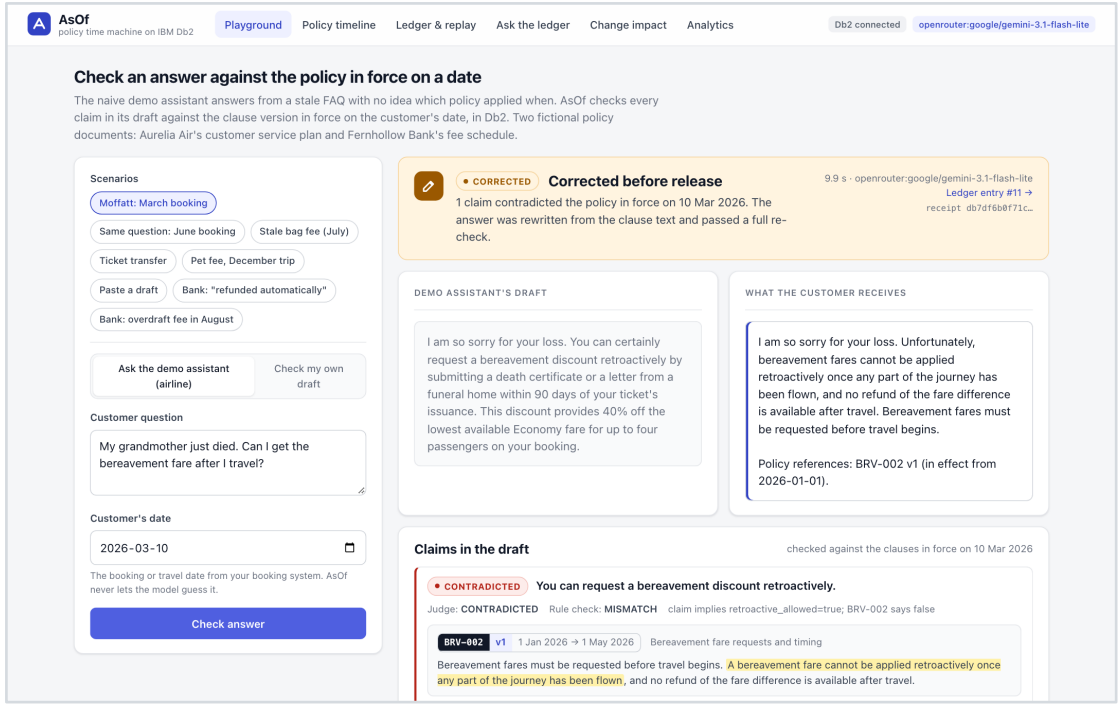
No evidence when it counts

In a dispute, teams cannot show which policy version an answer relied on, or what their systems knew at that moment.

AsOf: the policy in force on the customer's date

Given an AI-generated answer and a relevant date, AsOf finds the applicable policy version, flags claims that contradict it, stops unsupported answers from going out, and keeps the evidence needed to review that decision later.

- **As-of lookup.** Db2 returns the clause versions in force on the customer's date.
- **Claim check.** LLM judge, verbatim quotes, rules on rule_json, and every number must reach a claim.
- **Decision.** PASS with citations, CORRECTED and re-checked, or ESCALATED. Fails closed.
- **Audit ledger.** Hash-chained Db2 record of every decision, replayable.
- **Change impact.** Past answers and help-centre content a policy change makes stale.



The playground on a March booking: the demo assistant's retroactive promise is corrected from clause BRV-002 v1 before it reaches the customer.

How one answer flows through AsOf

1

Answer in

Draft, question and the customer's date from the calling system. The model never picks the date.

2

Extract claims

Structured output: up to 8 atomic policy claims.

3

Retrieve as of date

One Db2 query: FOR BUSINESS_TIME AS OF plus VECTOR_DISTANCE.

4

Judge and check

One batched judge call; an agent searches Db2 if a claim matched nothing. Verbatim quotes; rule_json overrides.

5

Decide

PASS, CORRECTED after a full re-check, or ESCALATED. Unchecked numbers or errors: held.

6

Record

APPEND_LEDGER writes a hash-chained row with citations and redacted text.

10 Mar 2026	*"You can claim the bereavement fare after you travel"*	**BRV-002 v1: request before travel**	**CORRECTED**
2 Jun 2026	*"You can claim the bereavement fare after you travel"*	**BRV-002 v2: retroactive within 90 days**	**PASS**

Same question, same words, different dates: the answer is right or wrong depending on which version was in force.

Architecture

Any AI assistant

Demo: a naive bot on LangChain with a DB2VS FAQ store

AsOf API (FastAPI)

Strict Pydantic validation, rate limit, security headers

Web UI

Playground, timeline, ledger and replay, Ask the ledger, change impact, analytics

Check pipeline

LangGraph check graph with an investigating agent (Db2 tools), deterministic rule and coverage checks, a change-impact agent, and a LangChain SQL chain for auditors. Models: OpenRouter gemini-3.1-flash-lite, falling back to local Ollama per call. Embeddings: MiniLM, local or in Db2.

IBM Db2 12.1.5, schema ASOF

Users: asofapp (runtime), asofedit (policy), asofread (analyst, masked), db2inst1

Policy

POLICY_DOC, POLICY_CLAUSE + history: bitemporal, VECTOR(384), RULE_JSON

Ledger

ANSWER_LEDGER, CLAIM_CITATION, CHAIN_HEAD: append-only, hash-chained

Content

PUBLISHED_ANSWER (help centre), FAQ vector store (DB2VS)

Routines and controls

6 SQL PL procedures, column masks (RCAC), audit views, a VANN vector index

Agentic AI: a LangGraph check and two agents, built

BUILT

LangGraph

- The check is a graph with fixed edges: extract, retrieve, judge, investigate, decide, rewrite
- Investigate: an agent whose tools query Db2 (as-of search, clause versions), at most 3 calls
- Change-impact agent: drafts a fix per stale help-centre answer, runs the full check, waits for the editor
- Per-call fallback from OpenRouter to local Ollama, recorded in the ledger

BUILT

LangChain and IBM's tools

- Ask the ledger: a SQL chain over SQLAlchemy (ibm_db_sa), every query gated by SQLGlot's Db2 dialect
- langchain-db2 DB2VS, a custom retriever, and a drop-in Runnable for any chain
- dbt marts (ibm-dbt-db2), an Airflow nightly audit (IBM Db2 provider), a Langflow component

Safety properties no agent step can bypass

Fixed graph edges | **Rules override the model** | **Every number must reach a claim** | **Full re-check before release** | **Any error means ESCALATED**

Innovation: adding time to answer checking

AsOf does not claim to have invented policy checking. What is new is time: what was true on date X, what our database knew at time T, and which past answers a change makes stale.

Closest existing tool	What it does	What AsOf adds
Amazon Bedrock Automated Reasoning checks	Validates answers against policies, suggests corrections, logs results	Effective-dated versions, replay of what the database knew, change impact, inside your own Db2, open source
NVIDIA NeMo Guardrails fact-checking rails	Scores an answer against retrieved chunks	Policy versions and dates, plus deterministic checks on structured rule parameters
IBM watsonx.governance	RAG quality metrics such as faithfulness	Per-answer evidence: which clause version, quoted, in a tamper-evident record



Effective-dated

Every clause version carries the dates it applied.



Bitemporal replay

FOR SYSTEM_TIME AS OF shows what the database held at answer time.



Change impact

A policy change lists the past answers and articles it makes stale.

Db2 integration: the database does the hard work

Bitemporal tables

BUSINESS_TIME and SYSTEM_TIME periods, a history table, a WITHOUT OVERLAPS key.

FOR PORTION OF

A policy change splits the version row; the old row moves to history.

VECTOR and VANN

VECTOR(384) with VECTOR_DISTANCE in the temporal query; a native vector index, benchmarked.

SQL PL procedures

Six, from APPEND_LEDGER with an in-SQL SHA-256 chain to CHANGE_IMPACT.

In-database AI

CREATE EXTERNAL MODEL and TO_EMBEDDING embed clauses inside Db2 (12.1.5).

SQL/JSON

JSON_EXISTS on every rule; JSON_TABLE in views and the citation shredder.

Column masks (RCAC)

Ledger text reads [masked] to everyone but the runtime user.

Least privilege

Three users and two roles; triggers block UPDATE and DELETE, even for the owner.

Connectors: ibm_db with bound parameters, langchain-db2 DB2VS, SQLAlchemy with ibm_db_sa, IBM's dbt adapter, Airflow provider and lfx-ibm.
Proof: python -m asof.db2check runs 29 of 29 Db2 feature checks; CI runs them on IBM's stock image on every push.

One query answers: what applied on that date?

```
SELECT c.CLAUSE_ID, c.VERSION_LABEL, c.CLAUSE_TEXT,
       c.RULE_JSON,
       VECTOR_DISTANCE(c.EMBEDDING,
         VECTOR(?, 384, FLOAT32), COSINE) AS DIST
FROM   POLICY_CLAUSE
       FOR BUSINESS_TIME AS OF ? AS c
WHERE  c.DOC_ID = ?
ORDER  BY DIST
FETCH  EXACT FIRST ? ROWS ONLY
```

Temporal filter and vector ranking in one statement. Every value is a bound parameter.

Example: raise the pet fee to $125 from 1 Dec 2026

Nov answer: still valid

Dec answer: stale

Help article: review

Replay

Add FOR SYSTEM_TIME AS OF the answer's timestamp and Db2 returns exactly what it held when the bot answered, even if the policy was edited later.

Change impact

APPLY_POLICY_CHANGE updates FOR PORTION OF BUSINESS_TIME. CHANGE_IMPACT then lists each past answer as STALE or HISTORICALLY VALID, and each help article that needs review.

Feasibility: built, tested and measured

0 of 64

wrong drafts released, four frozen splits, three runs

2 of 34

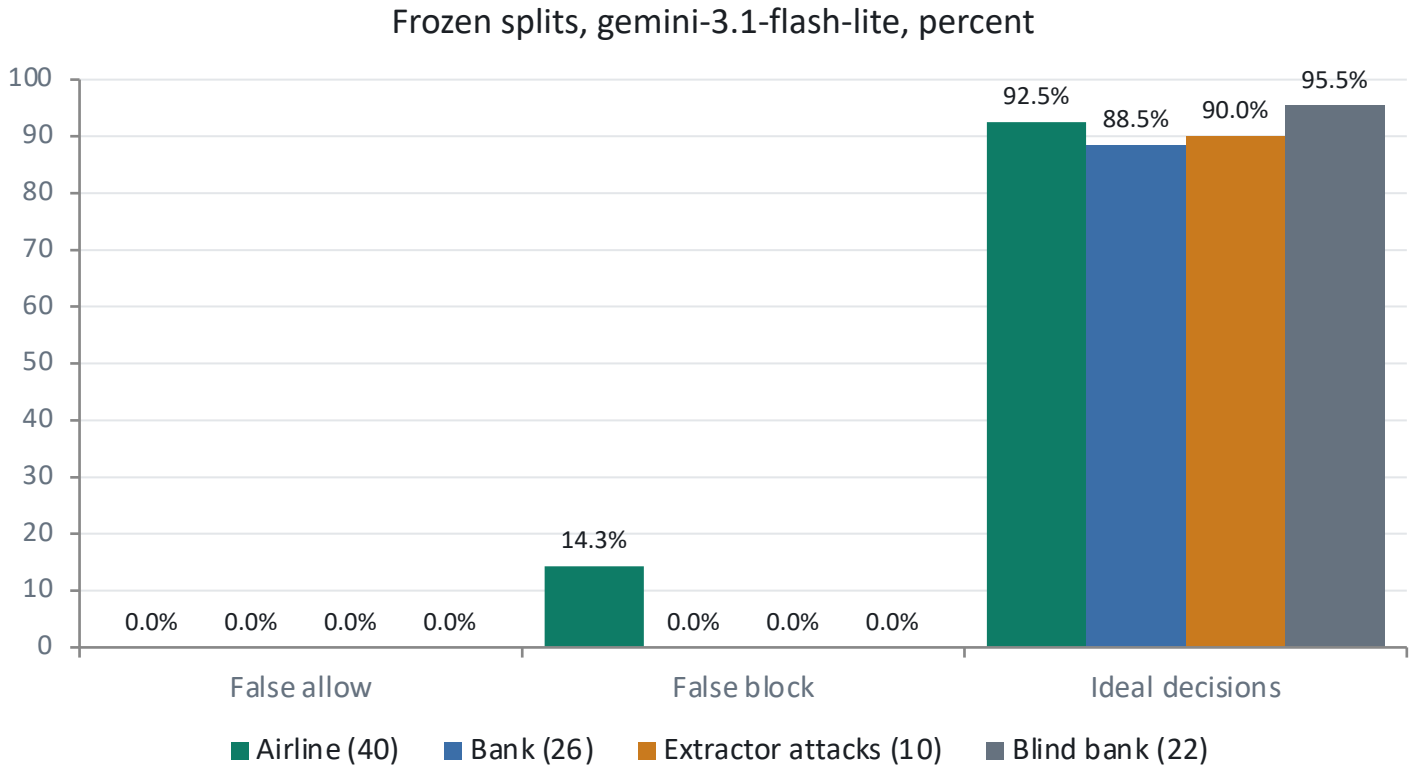
correct drafts held back in the latest run

98

frozen cases: airline 40, bank 48, extractor attacks 10

479

tests; 29 of 29 Db2 checks; three CI jobs, green



Each split was committed and hash-pinned before its first run; latest run at 397c920. The local Ollama model released 1 of 26 wrong airline drafts. ./quickstart.sh needs no API key: replay mode runs the built-in scenarios with Db2 live.

Security and trust by design

Fail closed

Any error, timeout or invalid model output becomes ESCALATED. No ledger record, no release.

Least privilege

Three users, two roles. Writes only through procedures. The analyst sees six views, text masked.

No string-built SQL

Bound parameters, enforced by an AST build check. Generated SQL runs only after a SQLGlot gate.

Prompt injection

Delimited data, strict schemas, rule overrides, and a coverage check behind the extractor.

Redaction and masks

Personal data redacted before storage; Db2 column masks on ledger text.

Tamper evidence

Triggers block UPDATE and DELETE; VERIFY_CHAIN finds the first altered row; receipts catch a rebuilt ledger.

Expected impact

For CX platform, compliance and legal-ops teams at airlines, banks, insurers and telcos: organisations that already run Db2.



Before release

Wrong policy promises are corrected from the clause text, or held for a specialist, with citations.



In a dispute

An evidence pack: the exact policy version, what the database knew then, and a receipt check.



Around a policy change

Preview the impact first; then every past answer and help article it makes stale is listed.

How it gets adopted: one REST endpoint, a LangChain Runnable or a Langflow component in front of any assistant. Runs on Db2 Community Edition; a second domain (a bank fee schedule) reused the schema, check and ledger, adding prompts, data and rule cues. Apache-2.0.

Aurelia Air and Fernhollow Bank are fictional; the airline is modelled on the policy at issue in Moffatt.

Status and next steps

Built and measured

- LangGraph check and two agents, fail closed
- Bitemporal, vector, JSON, SQL PL, RCAC, in-DB AI
- Hash-chained ledger, receipts, evidence packs
- Ask the ledger as a masked read-only analyst
- Two domains; four frozen splits; 479 tests
- dbt, Airflow, Langflow; quick start, no key

Next

- Anchor the chain head for rows without receipts
- Per-user authentication in front of the API
- Fewer safe escalations, measured on a new split
- Redact names, not only numbers and emails
- TLS to a remote Db2 such as Db2 on Cloud
- Contribute the recipe to IBM's AI cookbook

github.com/JustJay7/asof

Jay Malhotra | IBM Db2Hack 2026